Information Retrieval and Web Analysis

Final Project PART 2

Indexing and Evaluation

Serena El-Hajj &Souad Ibrahim El Husseini& Guillermo

November 3,2026

Tag : IRWA-2026-part-2

Github: https://github.com/Souad-El-Husseini/irwa-search-engine

1 PART 1 : Indexing

Introduction :

A dataset containing 28080 fashion products is between our hands. In this section we present the development of our indexing and ranking system.

We build on the preprocessing of the previous part of the project and our objective was to create an efficient retrieval system capable of identifying products according to a user's search query.

We first constructed a field-aware positional inverted index that preserves information about the fields and positions in which each term appears.

Secondly, we implemented conjunctive (AND) retrieval to ensure that all query terms are present in the retrieved products.

Finally, we adapted the TF-IDF ranking approach introduced in the practical sessions which ranks matching products according to their relevance to the query.

AI Use

- *ChatGPT helped us understand and improve our pseudocode. We adapted the practical lab code to our dataset ourselves.*
- *We adapted the lab code to implement AND search, using ChatGPT mainly for debugging.*
- *AI helped us fix errors, check outputs, and improve code readability.*

1.1 Inverted Index Construction

1.1.1 Dataset and Preprocessing

We first loaded the original fashion product 28080 products dataset (fashion_products.csv) along with the preprocessed dataset (index_info.csv) that we generated in the previous part of our project.
We retained the original dataset to retrieve product information (titles,brands, IDs).
The preprocessed dataset was used to construct our inverted index.
For indexing, we considered: **title, description, brand, category, sub_category, seller, and product_details** as our seven textual fields.
Finally, when it was necessary he preprocessed tokens stored in the CSV file were converted back into Python lists while missing or invalid fields were skipped.

1.1.2 Field-Aware Positional Inverted Index

After consideration, we decided to extend the basic structure of the inverted index approach introduced in the first practical session to a field-aware positional inverted index.
Instead of associating each term only with the products containing it, our index also records the field in which the term appears and its position within that field.

The resulting structure looks as such:

To construct the index, we iterate through each product and its seven searchable fields. For each of the latter, we identify the terms and record their positions, starting from zero. We then add these postings to the corresponding term and field in the inverted index.

For example, in the product Women Kurta and Pyjama Set Cotton Blend, the term pyjama appears at position 2 in the title and position 3 in the product details.

```
Term: women
Field: title, Positions: [0]

Term: pyjama
Field: title, Positions: [2]
Field: product_details, Positions
```

This approach supports term-frequency calculations and allows us to distinguish between matches occurring in different product fields.

1.2 Query Processing and Document Retrieval

1.2.1 Query Preprocessing

Following the same general approach used in part 1, the user's query is preprocessed before product retrieval.

We implemented a build_terms() function that:
-Converts queries to lowercase
-Performs tokenization
-Removes English stop words,
-Applies Porter stemming.
This ensures that query terms are represented consistently with the terms stored in our inverted index.

For example, the query *"Blue navy jeans for women"* is transformed into ['blue', 'navi', 'jean', 'women'].

1.2.2 Conjunctive (AND) Retrieval

We implemented a conjunctive retrieval system, as we were asked to do in the project requirements. Following this condition all query terms must appear in order to retrieve a product.

The search() function we implemented considers all seven indexed fields and uses the inverted index to identify the product IDs associated with each query term.
The sets of matching product IDs are then intersected to retain only products containing every query term.

Our function returns an empty list under either of these two cases:
-If a query term is absent from the index
-If no products satisfy all the conditions

The advantage of this approach is that only products satisfying the AND condition are considered for TF-IDF scoring which then reduces the number of candidate products prior to ranking

1.2.3 Query Selection and Refinement

To test our retrieval system , we initially proposed five *creative queries* as we were asked to do.
Our initial queries were:

Query 1: Women grinch printed pyjamas
Query 2: Blue navy jeans for women
Query 3: Men dotted tie
Query 4: Women long sleeve silk dress
Query 5: Men brown winter pants

However, several of these queries returned no matching products because they were too restrictive (*eg:* long sleeve) or contained terms that were absent from the indexed vocabulary(*eg:* grinch)

We therefore refined our original queries by removing some overly specific terms while preserving their main search intentions.

Our refined queries became:

Query 1: Women pyjamas
Query 2: Blue navy jeans for women
Query 3: Men dotted tie
Query 4: Women silk dress
Query 5: Men brown pant

This allowed us to obtain matching products and subsequently test the TF-IDF ranking system

1.3 TF-IDF Computation

We implemented the Term Frequency-Inverse Document Frequency approach used in the practice session 1 to rank the retrieved products according to their relevance.
TF-IDF considers the frequency of a term within a product and the importance of this term across the entire collection.

1.3.1 Term Frequency (TF) and Normalization

calculate_tf() function determines how frequently each term appears in a product.

By using the positional inverted index, the raw term frequency is obtained by counting the occurrences of a term across all seven indexed fields.

For example, in the product *Women Kurta and Pyjama Set Cotton Blend*, the term women appears once, while *pyjama* appears twice.

Following the approach used in the practical sessions, we then applied Euclidean normalization to the term frequencies. The normalization factor is calculated using the frequencies of all indexed terms within each product.

The normalized term frequency is given by:

$$Normalized\ TF(t,d) = f(t,d)\frac{f(t,d)}{\sqrt{\sum f(u,d)^2}}$$

where f(t,d) represents the raw frequency of term t in document d, and the sum considers all indexed terms u in that document.

This normalization helps reduce the influence of differences in document length.

For our example product, the normalized frequencies of women and pyjama were 0.0685 and 0.1370, respectively.

1.3.2 Document Frequency (DF)

calculate_df() function determines the number of distinct products containing each indexed term.
As we have implemented it, out inverted index preserves multiple fields. That means a term may occur in several fields of the same product. Therefore to avoid counting the same product more that once the set of product IDS were used when calculating document frequencies.

1.3.3 Inverse Document Frequency (IDF)

After calculating document frequencies, we implemented the calculate_idf() function to measure the importance of each term across the collection.

We followed the formula introduced in the practice session, IDF is calculated as:

$$IDF(t) = \log \frac{N}{DF(t)}$$

where N represents the total number of products in the collection (28,080), and DF(t) represents the number of distinct products containing term t.

For example, we obtained an IDF of 0.7092 for *woman* and 4.3512 for *pyjama.*

These results illustrate how IDF assigns greater importance to less frequent terms. Indeed, *pyjama* appears in considerably fewer products than *woman*, it receives a higher IDF value and can contribute more strongly to the final relevance score. (The term woman appears in 13817 products whereas pyjama in only 362 products)

1.4 Document Ranking

1.4.1 TF-IDF Vector Representation

The rank_documents() function follows the vector-space approach introduced in the practice sessions. TF-IDF weights were calculated for both queries and products by multiplying normalized term frequencies by their corresponding IDF values.

$$w_{t,d} = TF_{norm}(t, d) \times IDF(t)$$

This same principle is applied to calculate the query term weights as well:

$$w_{t,q} = TF_{norm}(t, q) \times IDF(t)$$

This allows us to represent both queries and products using weighted terms, giving greater importance to terms that are less frequent across the collection.

1.4.2 Ranking Using the Dot Product

In order to rank the retrieved product, we computed the dot product between their TF-IDF weights and those of the query.

We only considered products satisfying the conjunctive (AND) retrieval condition and the resulting products were then sorted in descending order of their relevance scores.

$$Score(d, q) = \sum w_{t,d} \times w_{t,q}$$

Where:

- $w_{t,d}$: TF-IDF weight of term t in product d
- $w_{t,q}$:TF-IDF weight of term t in query d
- The sum is over distinct query terms.

1.5 Search Engine Results

To verify our retrieval and ranking system, we tested five queries. As explained before some initial queries were too restrictive and returned no results, so we refined them while preserving their original search intentions.

The following table presents the final queries and the number of products retrieved using conjunctive (AND) search.

Query	Retrieved Products
Women pyjamas	193
Blue navy jeans for women	43
Men dotted tie	3
Women silk dress	32
Men brown pant	56

The retrieved products were ranked using TF-IDF. Although the top results were generally relevant, some unrelated products appeared due to matches across different fields, highlighting limitations to be explored during evaluation.

Rank	Product ID	Product Title	Brand	TF-IDF Score
1	PYJFZMJN25GEGHQE	Women Pyjama (Pack of 1)	Bonnevil	8.2949
2	PYJFZMJNGEDZGHEZ	Women Pyjama (Pack of 1)	Bonnevil	8.2949
3	PYJFZMJNHZF3ZYNR	Women Pyjama (Pack of 1)	Bonnevil	8.2949
4	PYJFZMJNRQDEFBHB	Women Pyjama (Pack of 1)	Bonnevil	8.2949
5	PYJFZMJNAWFWGEGA	Women Pyjama (Pack of 1)	Bonnevil	8.2757
6	PYJFZMJNFPHYWZN7	Women Pyjama (Pack of 1)	Bonnevil	8.2757
7	PYJFZMJNGNZ6GMKZ	Women Pyjama (Pack of 1)	Bonnevil	8.2757
8	PYJFP3TARHZKVXJM	Women Pyjama (Pack of 1)	K	7.4285
9	PYJFP3TASZSPTGRK	Women Pyjama (Pack of 1)	K	7.4285
10	PYJFP3TAZKYPYTRY	Women Pyjama (Pack of 1)	K	7.4285

For the query *"Women pyjamas*", the system retrieved 193 distinct products. The top 10 results correspond to the requested product type and are ranked by decreasing TF-IDF scores, indicating promising retrieval performance for this query.

2. Evaluation of the retrieval system

2.1 Evaluation setup and relevance judgments

We evaluated the Part 2 search engine using the instructor-provided validation queries and five custom fashion-product queries. The retrieval pipeline preprocesses the query, applies Boolean AND matching over seven indexed fields, and ranks eligible products with a TF-IDF weighted dot product. Each custom-query result was assigned a binary relevance label (1 = relevant, 0 = non-relevant) by inspecting the product information. The completed judgment file contains 327 query–product pairs: 137 labeled relevant and 190 non-relevant. ChatGPT assisted with reviewing some relevance decisions; the resulting labels should not be described as independently expert-assessed.

We set K = 10 consistently for all queries. Precision@10 counts ten ranking slots even when fewer than ten products were returned. For the two instructor queries, the provided ground truth covers only 40 judged items in total, so we evaluate a condensed ranked list of judged retrieved products, retaining judged-but-unretrieved relevant products in the relevance denominator. For

custom queries, recall is conditional on the labeled retrieved set, not all relevant products in the full catalog.

2.2 Results for the instructor validation queries

Query	P@10	R@10	AP@10	F1@10	NDCG@10
1 · women full sleeve sweatshirt cotton	1.000	0.769	1.000	0.870	1.000
2 · men slim jeans blue	0.600	0.600	0.549	0.600	0.706

The two instructor queries yield MAP@10 = 0.775 and MRR@10 = 1.000. Query 1 performs better across the judged evaluation pool: all ten considered top-ranked products are relevant, although the judged-pool recall is 0.769. Query 2 obtains lower precision and NDCG, indicating that non-relevant products appear among the higher-ranked judged results. The MRR of 1.000 means that the first judged retrieved result is relevant for both queries; it does not imply that all relevant products were found.

2.3 Results for the five custom queries

ID / search query	Matches	Relevant	P@10	R@10	AP@10	F1@10	NDCG@10
1 · Women pyjamas	193	76	1.000	0.132	1.000	0.233	1.000
2 · Blue navy jeans for women	43	6	0.600	1.000	0.931	0.750	0.978
3 · Men dotted tie	3	3	0.300	1.000	1.000	0.462	1.000
4 · Women silk dress	32	5	0.500	1.000	1.000	0.667	1.000
5 · Men brown pant	56	47	1.000	0.213	1.000	0.351	1.000

For the five custom queries, MAP@10 = 0.986 and MRR@10 = 1.000. Every query has a relevant product at rank 1. The high MAP indicates that judged relevant items are generally placed early in the results, under the evaluation's AP@10 convention (sum of precision at relevant ranks divided by min(K, number judged relevant)).

2.4 Interpretation of the evaluation metrics

- Precision@10 measures the proportion of the first ten positions occupied by relevant products. Women pyjamas and Men brown pant both achieve 1.000, so their first ten products are relevant. By contrast, Women silk dress achieves 0.500 because only the first

five are relevant. Men dotted tie achieves 0.300 because only three products match, even though all three are relevant.

- Recall@10 measures how many judged relevant products appear in the first ten ranks. For Women pyjamas, 10 of 76 judged relevant matches occur in the top ten, giving 0.132. For Men brown pant, the corresponding value is 10/47 = 0.213. These low values reflect the cutoff and the number of relevant retrieved products; they should not automatically be interpreted as poor ranking.
- Average Precision@10 rewards placing relevant products early. The score is 1.000 for four queries; Blue navy jeans for women is slightly lower at 0.931 because relevant products occur at ranks 1, 2, 3, 4, 6, and 8, with non-relevant results between them. MAP@10 is the mean of the five AP@10 scores.
- F1@10 balances precision and recall. It is 0.233 for Women pyjamas and 0.351 for Men brown pant: high top-ten precision does not offset their low recall at K=10. The strongest F1 is 0.750 for Blue navy jeans for women.
- NDCG@10 rewards highly relevant results appearing near the top. Its value is 1.000 for four custom queries because the relevant products available within the cutoff occupy the earliest positions; Blue navy jeans for women scores 0.978, still close to the ideal discounted ranking.
- MRR@10 equals 1.000 because the first returned product is relevant for all five queries. MRR measures the position of the first relevant result, not the completeness of the result set.

2.5 Precision–recall curves

Figures 1–5 plot precision against recall after each retrieved rank for each custom query. Every point is calculated directly from the completed relevance-label CSV and is therefore reproducible. Unlike the summary metrics, which use K = 10, each plot follows the entire retrieved list. A highlighted point marks rank 10 when that rank exists.

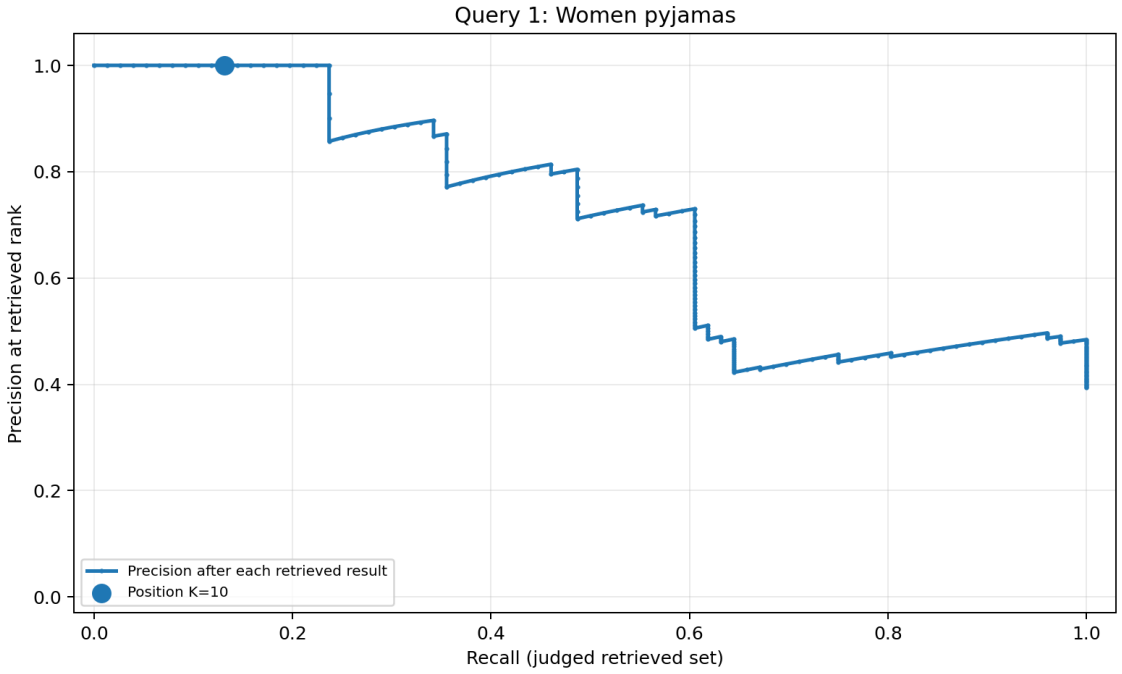
The curves for Women pyjamas and Men brown pant begin at precision 1.000 but eventually lose precision when non-relevant products appear later. Blue navy jeans for women and Women silk dress reach conditional recall 1.000 once their last judged relevant retrieved product is encountered; precision subsequently decreases as more non-relevant results are considered. Men dotted tie has only three retrieved products, all relevant, so its curve stays at precision 1.000; the separate P@10 metric remains 0.300 because the other seven positions are empty.

All five plotted recall curves reach 1.000 by the end because their denominator counts only relevant products among the retrieved and judged results. This does not demonstrate that the search engine found every relevant product in the complete catalog.

2.6 Limitations and possible improvements

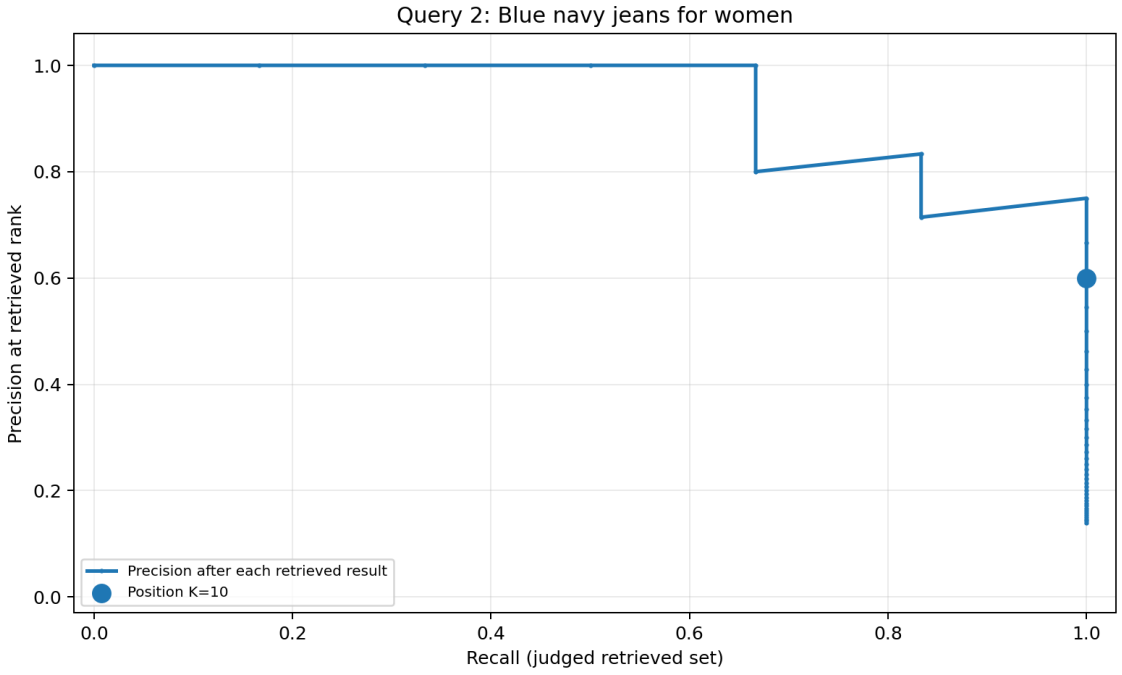
- Incomplete relevance ground truth: the instructor labels cover a small judged pool, while the custom labels cover only retrieved products. A larger independently assessed pool, including unreturned products, would support a stronger full-corpus recall estimate.
- Strict Boolean AND retrieval can exclude useful products that express the same need with synonyms or omit one query term. Query expansion, stemming checks, or a relaxed OR candidate stage could be evaluated for improved coverage.
- The ranking applies no field-specific weights: a term in the title is not explicitly distinguished from one in a long description or seller field. Field-aware weights could be tested using held-out judgments instead of assigned without validation.
- Metadata can be repetitive or noisy, and no structured filters are applied for price, rating, or availability. Cleaner attribute extraction and user-controlled filters could improve precision and usefulness.
- The experiment uses only five custom queries and K = 10. Evaluation across more varied user queries and multiple cutoff values would better establish how the method generalizes. BM25 and semantic retrieval can serve as baselines in later parts of the project.

Precision–recall figures



AND matches: 193 | Relevant: 76 | P@10: 1.000 | R@10: 0.132 | AP@10: 1.000

Figure 1. Precision–recall curve for custom query 1: "Women pyjamas".



AND matches: 43 | Relevant: 6 | P@10: 0.600 | R@10: 1.000 | AP@10: 0.931

Figure 2. Precision–recall curve for custom query 2: "Blue navy jeans for women".

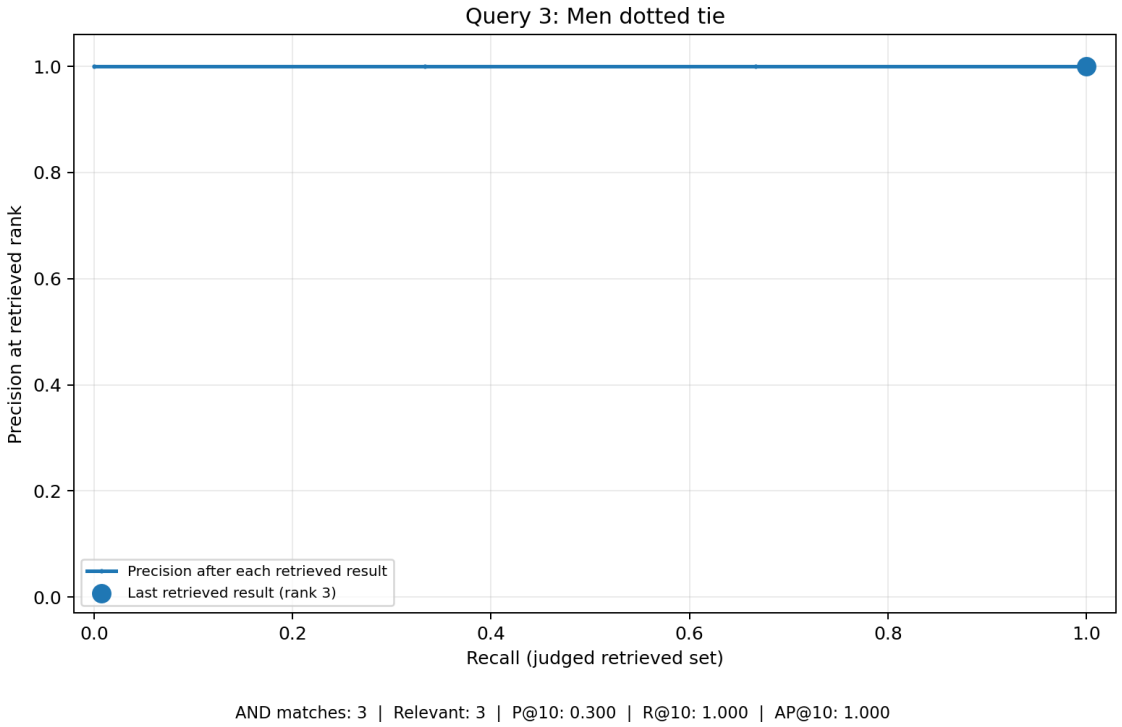


Figure 3. Precision–recall curve for custom query 3: "Men dotted tie".

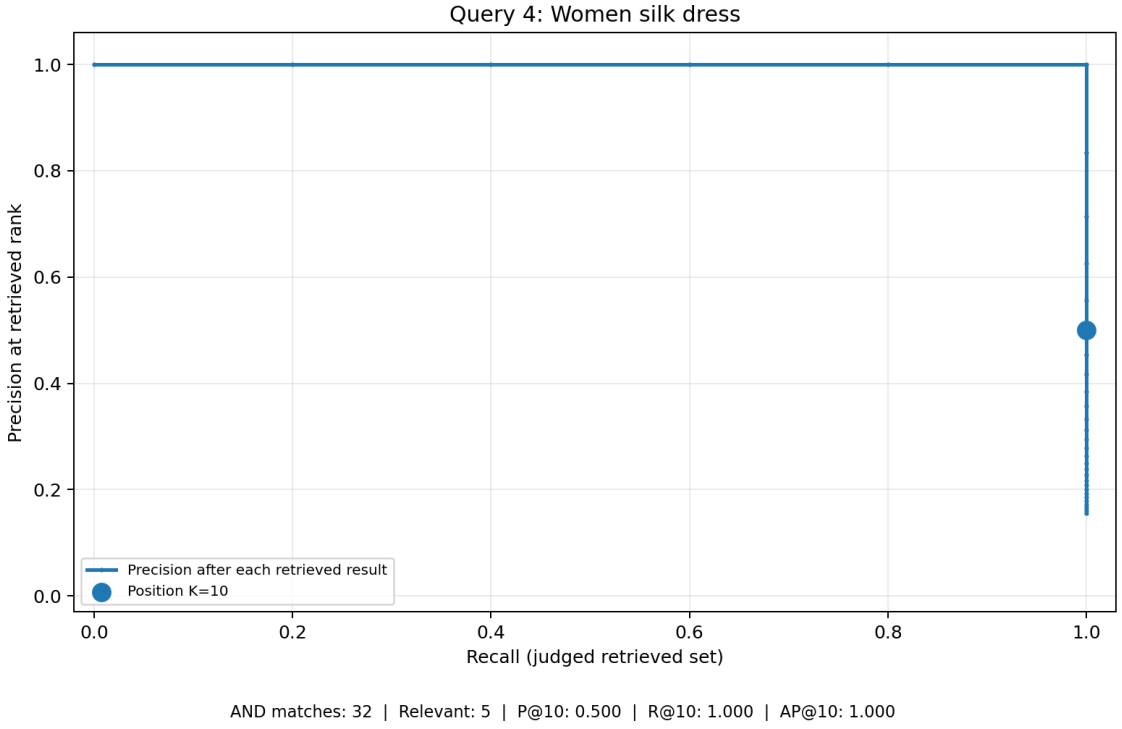


Figure 4. Precision–recall curve for custom query 4: "Women silk dress".

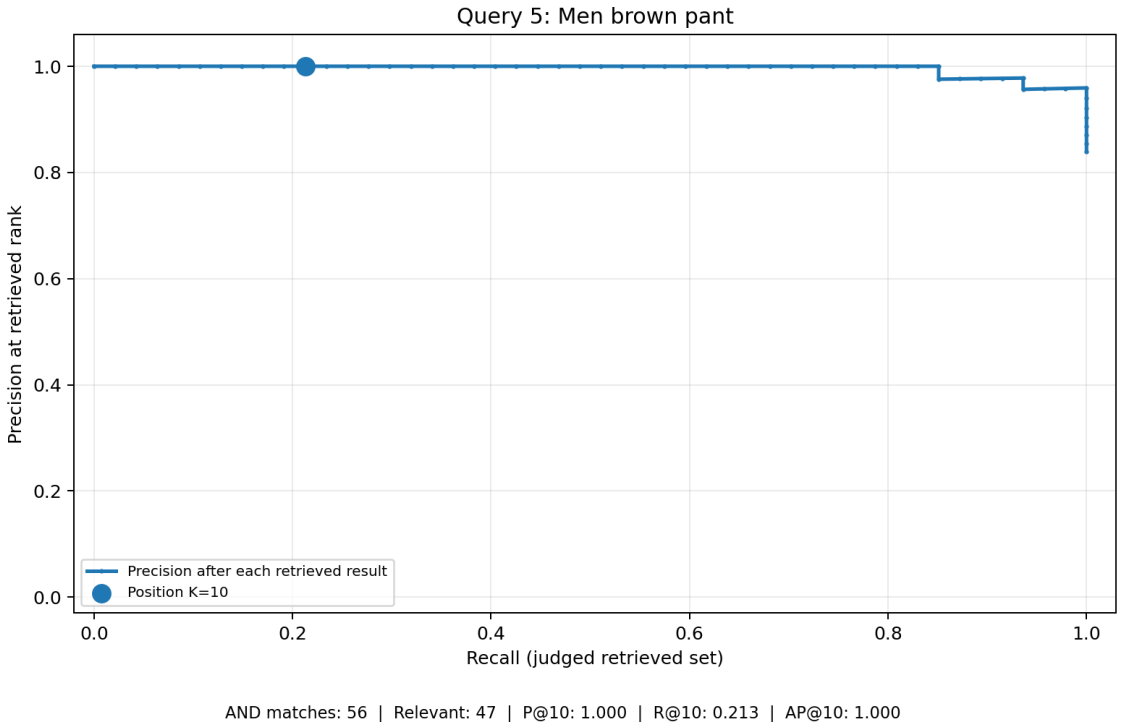


Figure 5. Precision–recall curve for custom query 5: "Men brown pant".